

Séance 2 — Exercice : le module Réservation

KFOKAM48 — Batch 2 — Phase 3 • Séances Full Stack

Le contexte

Dans la bibliothèque, un livre déjà emprunté ne peut pas être emprunté par quelqu'un d'autre. On veut permettre à un adhérent de le **réserver** : il sera prévenu dès qu'un exemplaire revient.

Vous ajoutez ce module au projet **Gestion de Bibliothèque**, sur lequel vous avez travaillé en séance 1. Vous partez de **main** à jour.

Aucun code ne vous sera donné. Le projet existant est votre modèle : lisez comment les autres entités sont écrites, et faites de même.

L'entité à créer

Reservation

Champ	Type	Contraintes
id	identifiant	généré
livre	relation vers Livre	obligatoire
adherent	relation vers l'entité utilisateur du projet	obligatoire
dateReservation	date et heure	générée par le serveur, jamais fournie par le client
dateExpiration	date et heure	calculée, voir RG-04
statut	énumération	EN_ATTENTE, DISPONIBLE, ANNULEE, EXPIREE, HONOREE

Les règles de gestion

Réf.	Règle
RG-01	On ne peut réserver qu'un livre indisponible . Réserver un livre disponible est refusé.
RG-02	Un adhérent ne peut avoir qu'une seule réservation active sur un même livre.
RG-03	Un adhérent ne peut pas dépasser 3 réservations actives simultanées.
RG-04	$dateExpiration = dateReservation + 7 \text{ jours}$. Calculée côté serveur.
RG-05	Une réservation ne peut être annulée que si son statut est EN_ATTENTE ou DISPONIBLE.
RG-06	Une réservation ANNULEE, EXPIREE ou HONOREE ne peut plus changer d'état.

Une réservation est dite **active** si son statut est **EN_ATTENTE** ou **DISPONIBLE**.

Les endpoints attendus

Verbe	Chemin	Rôle	Succès	Erreurs
POST	/api/reservations	Créer une réservation	201	400, 404, 409
GET	/api/reservations	Lister, filtrable par statut et par adhérent	200	—
GET	/api/reservations/{id}	Consulter	200	404
PATCH	/api/reservations/{id}/annuler	Annuler	200	404, 409
DELETE	/api/reservations/{id}	Supprimer	204	404

Le client n'envoie que `livreId` et `adherentId`. Tout le reste est déterminé par le serveur.

Ce qui est exigé

Architecture

Respectez le découpage du projet existant : entité, repository, service, contrôleur. **Aucune logique métier dans le contrôleur.**

DTO

L'entité `Reservation` ne doit jamais sortir du service. Un DTO en entrée, un DTO en sortie.

Validation

Une requête sans `livreId` ou sans `adherentId` est refusée en **400**, avec un message qui dit lequel manque.

Gestion des erreurs

Un identifiant inconnu renvoie **404**. Une règle de gestion violée renvoie **409** avec un message qui **nomme la règle enfreinte** — pas « erreur serveur ».

Documentation

Les cinq endpoints apparaissent dans Swagger, avec leurs codes de retour.

Barème

Élément	Points
Entité, repository, service, contrôleur, DTO fonctionnels	15
Règles de gestion RG-01 à RG-06 correctement implémentées	12
Total	27

Cette répartition est celle de l'épreuve finale : **le CRUD vaut 15 points, la règle métier 12**. Un CRUD qui marche mais accepte de réserver un livre disponible perd presque la moitié des points.

Livrable

Une branche `feature/reservation-prenom-nom`, une Pull Request décrite, relue par un pair.

Dans la description de votre PR : la capture Swagger de vos cinq endpoints, et une phrase par règle de gestion indiquant où vous l'avez implémentée.

Si vous finissez en avance

- Écrire un test unitaire sur RG-03 (limite de 3 réservations)
- Ajouter un endpoint qui liste les réservations expirées
- Faire passer automatiquement en **EXPIREE** les réservations dont la date est dépassée